Integrating Discrete-Event Simulation and Multi-Agent Reinforcement Learning for Dynamic Staff Allocation and Reduced Waiting Times in Hospitals

This research, introduces a novel framework for optimizing hospital operations. It integrates **Discrete-Event Simulation (DES)** with **Multi-Agent Reinforcement Learning (MARL)** to dynamically manage staff allocation in complex, stochastic hospital environments.

The model was rigorously validated using real clinical data from the **MIMIC-IV database**, processing 786 patient admissions to accurately simulate realistic scenarios. The core challenge addressed is the dynamic allocation of hospital staff under fluctuating operational conditions, aiming to enhance efficiency and patient care.

Harishankar Somasundaram, MSc in Artificial Intelligence

DBS Dublin Business School

The Healthcare Resource Allocation Problem: A Complex Challenge

Traditional static staffing models in hospitals often fail to cope with the inherent variability of patient arrivals and demand patterns. This leads to inefficiencies, increased patient wait times, and suboptimal resource utilization.

- **Patient Wait Times:** Directly influenced by staffing levels, bed occupancy, and variable patient flow through different departments.
- **Non-linear Interdependencies:** Bottlenecks in one department can have cascading negative effects on downstream capacity and overall hospital flow.

System aims to address these issues by focusing on:

- **Dynamic Staff Allocation:** Adapting staffing levels in real-time.
- **Wait Time Minimization:** Reducing patient waiting periods.
- **Throughput Optimization:** Increasing the number of patients served efficiently.
- **Cost-Efficiency:** Optimizing resource use without compromising care quality.

The model incorporates **9 distinct hospital departments**, including Emergency Department, ICU, Medicine, and Discharge Lounge, capturing a comprehensive operational landscape.

Made with GAMMA

System Architecture: A Layered Approach

Configuration Layer

`config.py`: Defines essential constants, hyperparameters, and curriculum definitions for training.

Data Layer

`data_loader.py`: Manages MIMIC-IV data ingestion, patient pathway construction, and acuity computation.

Environment Layer

`hospital_env.py`: Implements the core hospital simulation engine, adhering to the Gym interface.

Algorithm Layer

`maddpg.py` and `mappo.py`: Contain the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) and Multi-Agent Proximal Policy Optimization (MAPPO) algorithms.

Training Orchestration

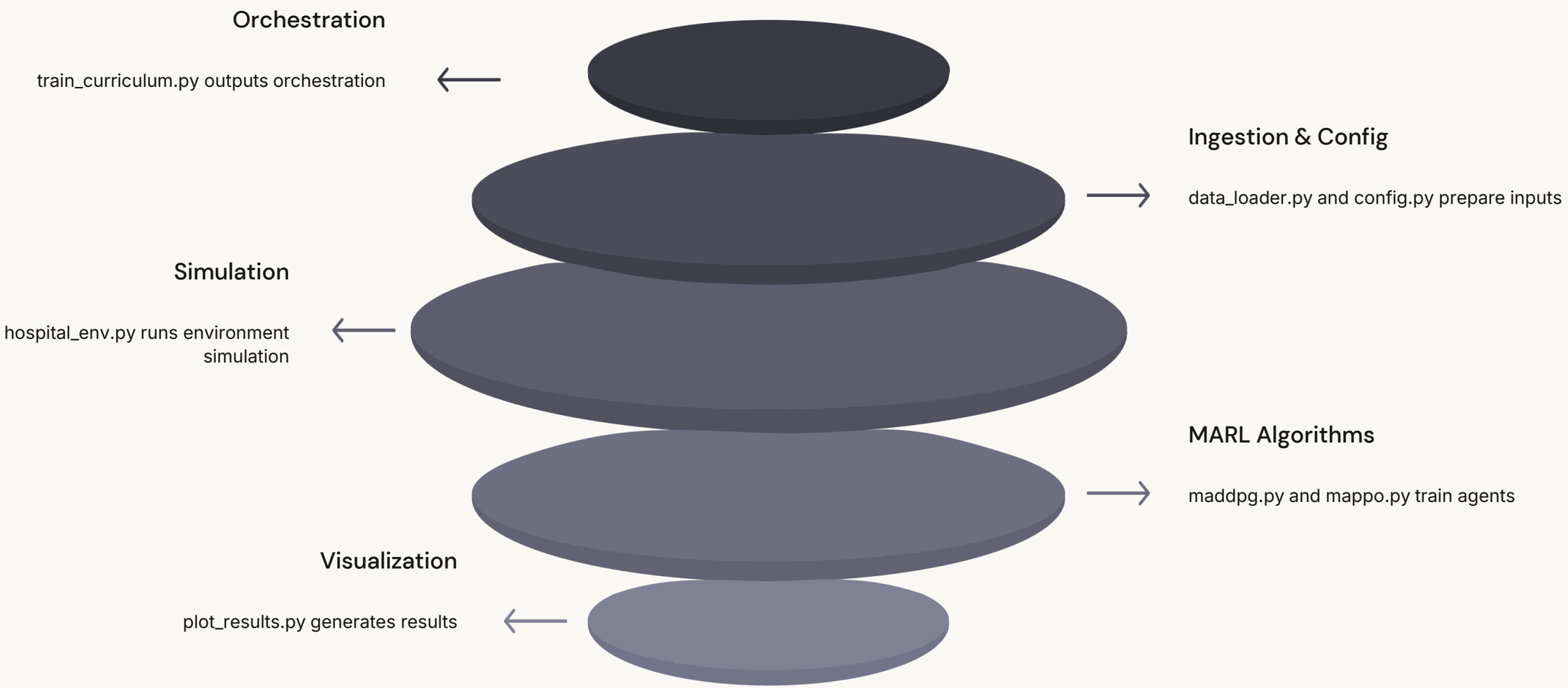
`train_curriculum.py`: Manages curriculum learning stages, logs experimental data, and handles model checkpointing.

Visualization Layer

`plot_results.py`: Generates insightful visualizations of training curves and algorithm performance comparisons.

Made with GAMMA

System Architecture: Component Flow



Made with GAMMA

Data Sources and Preprocessing: Leveraging MIMIC-IV

The foundation of this research relies on the **MIMIC-IV (Medical Information Mart for Intensive Care IV)** database, a comprehensive, de-identified clinical dataset. This ensures patient privacy while providing rich, real-world data for simulation.

Key Data Files:

- `admissions.json`: Contains detailed patient records, including admission timestamps and types (e.g., emergency, elective).
- `transfers.json`: Logs patient transfers between different care units and departments within the hospital.

Preprocessing Steps:

- **Pathway Construction:** Patient pathways are dynamically constructed by iterating through transfer records, mapping raw care units to standardized departments using a predefined `CAREUNIT_MAP`.
- **Acuity Computation:** Patient acuity is calculated based on admission type (e.g., EMERGENCY=0.85, ELECTIVE=0.4) with additional modifiers for ICU or Stepdown Unit admissions, reflecting higher care needs.
- **Validation:** Rigorous validation ensures data integrity. Pathways with fewer than two departments are excluded, and all processed pathways are confirmed to conclude with a transfer to the Discharge Lounge, simulating a complete patient journey.

Made with GAMMA

Internal Data Representation for Simulation and RL

Patient Structure

- **Acuity:** Numerical value representing patient severity.
- **Pathway:** Sequence of departments a patient will visit.
- **Pathway Index (`pathway_idx`):** Current position in the patient's pathway.
- **Current Department:** The department the patient is currently in.
- **Arrival Time:** Time patient arrived at the current department.
- **Wait Start:** Timestamp when patient began waiting in a queue.
- **Total Wait:** Accumulated waiting time.
- **Service Start:** Timestamp when patient began receiving service.

Environment State

- **Queues:** Patient IDs waiting in each department.
- **In Service:** Tuples of (patient ID, service completion time).
- **Staff Counts:** Current number of staff in each department.
- **Discharged List:** Records of discharged patients.

Observation Space (Per Agent)

- **12-dimensional vector:** Includes queue ratio, service utilization, normalized staff counts, wait time, acuity, episode progress, capacity, discharge ratio, and queue trend.

Action Space (Per Agent)

- **4-dimensional continuous vector:** Represents staffing deltas for doctors, nurses, healthcare workers (HCWs), and administrative staff.
- Actions are processed through a momentum-based accumulator for smoother staffing adjustments.

Made with GAMMA

Simulation Execution Flow: Modeling Hospital Dynamics

01

Episode Duration

Each simulation episode spans **168 hours (7 days)**, discretized into 5-minute timesteps, totaling **2016 timesteps** per episode.

02

Reset Phase

At the start of each episode, the environment state is cleared, an arrival schedule is built from historical data, arrival times are scaled, and initial staffing levels are set.

03

Per-Step Execution Loop

Each timestep involves sequential processing:

1. Application of agent actions (staffing adjustments).
2. Processing of new patient arrivals.
3. Processing of patients completing service.
4. Advancement of patient queues.
5. Computation of rewards for agents.
6. Generation of new observations for agents.

04

Service Time Calculation

Service time is determined by: `(base_time / staff_factor) × acuity_factor × lognormal_noise`. The `staff_factor` exhibits a logarithmic dependency on staff numbers, and a lognormal multiplicative noise (σ=0.15) introduces realistic variability.

05

Stochastic Elements

Realism is enhanced through stochastic elements:

- Arrival times are scaled from historical patient data.
- Service times incorporate lognormal multiplicative noise to reflect real-world unpredictability.

Made with GAMMA

State, Action, and Reward Design for RL Agents

Observation Space Features

Each agent receives a comprehensive observation vector, including:

- Queue length ratio (current queue vs. capacity)
- Service utilization percentage
- Current counts of 4 staff types (doctors, nurses, HCWs, admins)
- Average patient wait time in the department
- Average patient acuity
- Overall episode progress

Action Space

Agents produce continuous staffing adjustment signals in the range `[-1, 1]`. These signals are smoothed using a momentum-based accumulator to prevent rapid, destabilizing oscillations in staff allocation.

Reward Function Components

The reward function is meticulously designed to guide agent behavior towards optimal hospital operations:

- **Wait Penalty:** A quadratic penalty `-10.0 × (wait_minutes / 15)^2` heavily penalizes prolonged patient waiting times.
- **Queue Penalty:** A cubic penalty `-8.0 × 10.0 × (queue_ratio - 0.5)^3` is applied when a department's queue exceeds 50% capacity, discouraging severe backlogs.
- **Idle Capacity Penalty:** Penalizes underutilized staff resources.
- **Throughput Bonus:** Rewards the number of patients successfully discharged, capped to prevent reward hacking.
- **Staffing Cost Penalty:** Penalizes deviations from a baseline staffing level to encourage efficient resource use.

Made with GAMMA

MARL Algorithms and Curriculum Learning for Robust Policies

Multi-Agent Reinforcement Learning Algorithms

- **MADDPG (Multi-Agent Deep Deterministic Policy Gradient):**
  - Per-agent actors with (256-256) neural network architecture and `tanh` activation.
  - Centralized critic observes all agents' observations and actions, capturing interdependencies.
  - Soft target updates (`τ=0.01`) for stability.
  - Gaussian exploration noise to encourage diverse policy discovery.
- **MAPPO (Multi-Agent Proximal Policy Optimization):**
  - Actor-critic architecture with Gaussian policy.
  - Centralized value critic for stable learning.
  - Generalized Advantage Estimation (GAE) with `λ=0.90`.
  - PPO clipped objective (`ε=0.2`) to prevent large policy updates.
  - Entropy bonus (`0.05`) to promote exploration.

Five-Stage Curriculum Learning

Direct training of MAPPO policies without curriculum led to policy collapse between episodes 300-400. A structured curriculum was essential for robust learning:

1. **Stage 1:** Acuity range `0.0-0.60`, max pathway length `4`, patient fraction `35%`, staffing multiplier `1.5x`.
2. **Stage 2-4:** Gradually increases acuity range to `0.0-1.0`, max pathway to `10`, patient fraction to `100%`, and staffing multiplier to `1.0x` (normal staffing levels).
3. **Total Episodes:** Each stage comprises a set number of episodes (10, 15, 20, 25, 30), totaling 100 episodes across the entire curriculum.

Made with GAMMA

Results and Performance Analysis: Significant Improvements

Mean Wait Time	28.4 hours	2.0 hours	4.8 hours
Reduction	N/A	92.9%	83.1%
Throughput	306 patients	726.8 patients	450 patients
Improvement	N/A	137.5%	47.1%
Mean LOS	40.3 hours	15.9 hours	12.4 hours

The results demonstrate significant improvements over the baseline:

- **MADDPG** significantly outperformed the baseline, achieving a **92.9% reduction in mean wait time** and a remarkable **137.5% increase in patient throughput**. Its centralized critic effectively modeled complex interdepartmental relationships, leading to superior optimization.
- **MAPPO** also showed substantial improvements, reducing mean wait time by **83.1%** and increasing throughput by **47.1%**. While not as aggressive in throughput optimization as MADDPG, MAPPO demonstrated **superior preservation of Length of Stay (LOS)** with a mean LOS of 12.4 hours, likely due to its conservative policy updates from the clipped objective.

Made with GAMMA

Contributions, Limitations, and Future Extensions

Key Contributions

- Pioneering integration of **Discrete-Event Simulation (DES)** and **Multi-Agent Reinforcement Learning (MARL)** for hospital operations optimization.
- Demonstration of the critical necessity of **curriculum learning** for stable and effective policy training in complex healthcare environments.
- Rigorous **validation on real-world MIMIC-IV clinical data**, enhancing the applicability and credibility of the findings.

Modeling and Algorithmic Limitations

- **Simplified Staff Dynamics:** Current model does not account for complexities like hiring delays, staff fatigue, or varying skill sets.
- **Fixed Pathway Execution:** Patient pathways are assumed to be static; dynamic changes or unexpected complications are not modeled.
- **No External Events:** Excludes real-world disruptions like shift changes, holidays, or sudden emergencies.
- **Scalability of Centralized Critics:** Centralized critics in MADDPG can become computationally intensive with more agents/states.
- **Fixed Curriculum Stages:** The curriculum stages are predefined; adaptive curriculum methods could offer further improvements.
- **Reward Hacking Potential:** Complex reward functions can sometimes lead to agents finding unintended ways to maximize rewards.

Implementation Trade-offs

- **Momentum Accumulator Latency:** While beneficial for stability, the momentum-based action processing can introduce a slight delay.
- **Episode-Based Training:** The episodic nature of training limits real-time adaptability during continuous operation.
- **No Mid-Episode Checkpointing:** Lacks the ability to save and resume progress within a single simulation episode.

Future Extension Points

- **Expanded Department Coverage:** Integrate more hospital departments and inter-hospital transfers for a broader scope.
- **Advanced MARL Algorithms:** Explore newer, more scalable MARL algorithms, including those with decentralized execution capabilities.
- **Dynamic Reward Function Adaptation:** Implement mechanisms for dynamically adjusting reward function components based on operational goals.
- **Customizable Curriculum Stages:** Develop an adaptive curriculum learning framework that adjusts difficulty based on agent performance.
- **Alternative Data Sources:** Incorporate additional data streams (e.g., electronic health records, real-time sensor data) for richer simulations.

Made with GAMMA